AI-Based Intelligent Routing in Computer Networks

Kanzariya Nimesh
KU ID: KU2407U098
B.Tech. Computer Science

Kavya Patel
KU ID: KU2407U661
B.Tech. Computer Science

Nisarg Shah
KU ID: KU2407U151
B.Tech. Computer Science

UnitedWorld Institute of Technology, Karnavati University
Uvarsad, Gandhinagar, Gujarat 382422, India
Under the guidance of Pragya Vats | Mini Project, October 2026

***Abstract*—This paper investigates AI-assisted route selection under changing traffic loads and network failures using a flow-level simulator. The system constructs up to four candidate paths and scores them with a GATv2-based graph neural network that uses node, link and flow features. The evaluated checkpoint was transferred from behavior-cloning initialization and frozen before any online reinforcement-learning updates. A separate load-aware Dijkstra fallback handles specified invalid, unavailable or overloaded decisions, and the Python simulator and inference code are integrated with a FastAPI backend and a React/TypeScript interface. The saved final evaluation comprises 3,000 episodes over five designated test topologies, five scenarios, three traffic intensities and five seeds, comparing the AI policy with fallback against seven baselines. The supplied records were checked for consistency and the acceptance criteria were recalculated; simulations were not independently rerun. In the normal scenario, the AI policy with fallback reduced mean modeled loss relative to static Dijkstra by approximately 49.83%, 51.63% and 53.61% at intensities 0.05, 0.10 and 0.20, and therefore failed the criterion requiring a reduction of at least 50% overall. It passed the specified tolerance relative to load-aware Dijkstra, although its mean loss was higher than that baseline's in all 15 scenario–intensity groups. No accepted invalid actions or nonfinite valid-score decisions were recorded. The findings support a qualified, simulation-based assessment; earlier calibration involved two designated test graphs, which limits claims of untouched generalisation, and no physical packets were transmitted.**

***Index Terms*—intelligent routing, graph neural networks, GATv2, load-aware Dijkstra, fallback routing, network simulation, behavior cloning.**

I. Introduction

Routing determines the path that traffic follows through a network. A route may be selected using a fixed link cost or a cost that changes with network conditions. In Open Shortest Path First (OSPF), routers maintain a link-state database and calculate shortest-path trees; the protocol also responds to topology changes and supports equal-cost multipath routing [1]. Conventional routing therefore includes mechanisms for adaptation. The relevant question for this work is how a learned policy compares with specified routing rules under the same simulated conditions.

Research on graph neural networks provides a basis for representing relationships between network topology, traffic and performance. RouteNet, for example, models delay and loss using graph-structured information about topology, routing and input traffic [2]. It is related work that motivates studying graph-based learning for networking; its architecture and experimental findings are separate from this implementation.

This paper presents *AI-Based Intelligent Routing in Computer Networks*, a flow-level simulator with a learned candidate-path selector. The model uses GATv2, a graph attention architecture whose attention can depend on the query node [3]. Its parameters are initialized by transfer from a behavior-cloning checkpoint trained with a candidate-constrained load-aware routing policy as the expert. A separate fallback applies load-aware Dijkstra when specified checks reject the model's decision. The project also includes a web application for configuring network plans and accessing simulation results.

A. Problem Statement

The problem addressed is the selection of an available source-to-destination path when simulated link loads and network availability change. Each flow has a bandwidth demand, and multiple active flows can share links. In the implemented environment, increased load affects modeled delay, congestion loss and residual capacity; link or router failures can remove usable paths, and traffic spikes and bandwidth reductions change the conditions under which a route is selected.

The static Dijkstra baseline minimizes the sum of configured link latencies over the available graph. Its cost does not incorporate current traffic load, so it may keep preferring a low-propagation-delay path even when that path is congested; "static" describes the cost definition, as failed network elements are still excluded. The load-aware Dijkstra baseline uses a cost that incorporates current utilization and modeled latency. The two baselines allow the effect of load awareness to be examined separately from the use of a learned model.

The learned policy scores up to four candidate paths using features of the network and the requested flow. Restricting the action set makes selection explicit but limits the routes available to the model at each decision, whereas the fallback can search the full available graph and may return a route outside the candidate set. Performance attributed to the combined system must therefore account for both learned and fallback decisions. The work does not presume that the learned policy outperforms every baseline; evaluation considers modeled loss and latency, delivered demand, utilization, route changes and decision overhead, and identifies cases in which no usable path exists.

B. Aim and Objectives

The aim is to implement and assess AI-assisted path selection in a reproducible simulation environment, using explicit

network-state inputs and conventional routing baselines. The implemented system provides the following capabilities:

1) *Represent and validate network inputs.* Accept router and host definitions, link bandwidth, latency and configured loss, together with traffic inputs, and reject topologies or values that violate the model API's constraints.
2) *Construct routing observations.* Track simulated node and link conditions and transform network-state and flow features into model inputs.
3) *Select among candidate paths.* Generate up to four routes, mask unavailable actions and score the remaining candidates with the GATv2-based network.
4) *Apply a defined fallback.* Use load-aware Dijkstra when the wrapper detects an invalid or unavailable decision, nonfinite valid scores, or the specified overload condition, and record when and why it is used.
5) *Support controlled comparisons.* Simulate normal operation, link failures, router failure, traffic spikes and bandwidth drops, and produce metrics for comparison with baseline policies.
6) *Connect simulation to the application.* Provide backend interfaces for network plans and simulation runs, with a frontend for entering configurations and viewing aggregate results.

The assessment objectives are to determine where the combined policy helps, where it performs similarly to conventional methods and where it performs worse. Agreement with the expert policy is a separate diagnostic describing action matching; it does not by itself establish lower loss or latency, and successful execution of a routing decision does not establish a performance improvement. The repository also contains reinforcement-learning training mechanisms, including Double-DQN; however, the frozen inference checkpoint was selected at step 0 after transfer from behavior cloning, before online reinforcement-learning updates. This paper therefore distinguishes the implemented training framework from the learning history of the model actually used for inference. Demonstrating stable gains from further online learning remains a future objective.

C. Scope of the Project

The model API accepts connected, undirected topologies containing 5–50 routers; hosts must be leaf nodes with a single connecting link. Link inputs use Mbps for bandwidth, ms for latency and percentages for configured loss. These are interface constraints and do not demonstrate scalability to larger networks or deployment on physical hardware. The routing API selects a path for an individual flow, while the simulation API evaluates a sequence of flow decisions. Network monitoring means observing simulator state: loads, failures and scenario changes are represented in software, and the system does not capture router telemetry, install forwarding rules through an SDN controller or transmit packets between physical devices.

The simulation API compares static Dijkstra, load-aware Dijkstra and the AI policy with fallback. A broader evaluation harness adds random selection, utilization-based selection, an ECMP-style candidate selector and a random-forest baseline; these names denote the particular algorithms implemented here, not complete implementations of networking protocols. Comparisons must also distinguish candidate-constrained policies from policies that search the full graph. The simulator reports modeled path-loss probability, latency, delivered demand (Mbps), utilization and routing diagnostics; delivered demand is based on flow demand and available capacity rather than counted packets.

The frozen policy uses the current processed observation and ignores the history argument accepted by the API. The current simulation API also emits an AI decision trace (candidate paths, scores, chosen routes and fallback reasons) alongside aggregate metrics, but this does not establish that older saved runs contain traces, nor does it provide per-link heatmap histories or saved attention attribution. The reliability of a learned policy outside its training conditions also requires examination: imitation-learning research shows that a learner's actions influence the states it later encounters, creating a gap between expert-training data and operating conditions [4]. This motivates testing across traffic and topology conditions but does not establish generalisation to unseen operational networks. Physical-network validation, broader scalability testing, richer telemetry and further online learning remain outside the demonstrated scope.

II. System Requirements and Design

A. Hardware Requirements

The project runs as a software simulation on a general-purpose computer: network nodes are graph objects and traffic is represented by flows with specified demand and lifetime, so physical routers, switches and packet-capture equipment are not prerequisites. Project records describe a development environment with approximately 15.73 GiB of RAM and a historical configuration with 24 logical CPU cores. These observations are not established minimum requirements; the supplied evidence contains no controlled hardware-sizing study, minimum storage requirement or verified GPU requirement. Suitability must be judged against topology size, number of simulation steps and number of repeated policy evaluations, and training and single-decision inference have different computational demands. The complete application requires a computer able to run the Python services and JavaScript tools, a browser, storage for checkpoints and run outputs, and access to the configured database; a live database connection or deployment was not tested.

B. Software Requirements

The project specifies Python 3.12. The AI component uses PyTorch and PyTorch Geometric for the graph neural network, Gymnasium for the environment interface, NetworkX for graph operations and NumPy for numerical processing; Pandas, SciPy and scikit-learn support data processing and the broader evaluation tools. The backend uses FastAPI, Uvicorn and Pydantic, with SQLAlchemy, Alembic and the Psycopg driver for PostgreSQL/Neon persistence (SQLite appears only in test fixtures). The frontend uses React, TypeScript, Vite and

Tailwind CSS. Table I summarizes the declared dependencies; it is not an experimental-results table. A version range permits multiple package versions while a pin names one, and neither establishes which packages were installed for an older saved run. Versions were read from project files and installation compatibility was not independently verified; reproducing a saved evaluation also requires its checkpoint, configuration, topology, traffic settings and seeds.

TABLE I
SELECTED SOFTWARE DECLARATIONS IN THE SUPPLIED PROJECT

Component	Declared packages or versions	Role
AI runtime	Python 3.12; torch 2.14.1; torch-geometric 2.8.0.post1	Load and execute the learned path scorer
Simulation	gymnasium 1.4.0; networkx 3.7; numpy 2.5.3	Environment interface, topology operations, observations
Web backend	fastapi 0.161.1; uvicorn 0.54.0; pydantic 2.14.1	HTTP service, application execution, input validation
Persistence	SQLAlchemy 2.1.6; alembic 1.20.1; psycopg 3.3.6	Database access and migrations
Frontend	React ^18.3.1; TypeScript ^5.7.2; Vite ^6.0.7; Tailwind CSS ^3.4.17	Network configuration and result views

C. System Architecture

1) Components and Network Representation

The architecture connects a browser interface, an application backend and a routing simulator (Fig. 1). The frontend submits network and experiment settings; the backend translates them into the AI module's input representation, invokes routing or simulation, and persists results. Within the simulator, a monitor exposes link loads and operating conditions, a path generator constructs alternatives, and the learned scorer evaluates the processed observation. A separate validation wrapper can replace the model's choice with load-aware Dijkstra.

The model API validates a connected, simple, undirected graph with 5–50 routers; hosts must be leaf nodes, link bandwidth and latency must be positive, and configured loss is a percentage between 0 and 100. The environment itself has broader construction options, so its internal limits should not be confused with the API's validation rules. Automatic host attachment depends on the entry point: the individual `route()` call disables it, whereas the simulation path uses environment defaults, which can add hosts when the supplied topology has fewer than the target (normally the larger of four hosts and half the router count, rounded up). The example topology in Fig. 2 already has five routers and four hosts.

2) Observations and Preprocessing

The environment records node, edge and current-flow features, candidate routes and an action mask (Table II, reproduced unchanged from the project specification [8]). Here N is the number of nodes, E the number of undirected edges, K the candidate limit (four in this configuration) and T the history length.

TABLE II
OBSERVATION SPACES AND FEATURE MEANINGS (SOURCE TABLE UNCHANGED [8])

Field	Shape and dtype	Columns / meaning	Typical range
node_features	(N, 6), float32	[incident offered load / incident capacity, up, node_type, is_src, is_dst, degree / max_degree]	Load is ≥ 0 and may exceed 1; all other columns are in [0,1]. node_type is 0 for router, 1 for host. degree is in [0,1].
edge_features	(E, 5), float32	[offered-load utilization, latency_ms, loss_probability, up, capacity_Mbps]	Utilization ≥ 0 and may exceed 1; latency ≥ 0 and capped queue contribution; loss/up in [0,1]; capacity > 0 when up and zero after a failure.
flow_features	(7,), float32	[offered_demand, qos_REAL_TIME, qos_STREAMING, qos_BULK, qos_BACKGROUND, priority, remaining_duration_steps]	Demand ≥ 0; QoS entries are one-hot or all-zero without a flow; priority and duration are nonnegative.
candidate_paths	(K, N), int32	Candidate routes as node indices in NetworkRoutingEnv.nodes order.	Valid indices are [0, N−1]; unused path positions and rows are −1.
action_mask	(K+1,), int8	Valid candidate actions followed by sentinel action K.	Binary.
edge_index	(2, E), int32	Each column is the endpoint node indices for the corresponding edge_features and history_edges row.	Indices in [0, N−1].
history_nodes	(T, N, 6), float32	Past/current node feature snapshots, oldest first. Includes static metadata and each snapshot's source/destination markers.	Same ranges as node_features.
history_edges	(T, E, 5), float32	Past/current edge feature snapshots, oldest first.	Same ranges as edge_features.

Table II describes the raw environment interface. `ObservationWrapper` implements preprocessing that older documentation describes as future work: it duplicates undirected edges in both directions for message passing, clips edge utilization to [0, 2], transforms latency logarithmically and scales capacity by a fixed initial maximum; flow demand is scaled by median backbone capacity and duration by 200. It also computes six candidate-path features: hop count, log-transformed total latency, residual-capacity-to-demand ratio, maximum utilization, mean utilization and maximum backbone utilization (backbone links connect routers; host-access links are distinguished). The history arrays remain part of the interface, but the frozen policy uses only the current observation and has its recurrent GRU option disabled.

3) Candidate Generation and Learned Scoring

Candidate generation removes failed nodes, failed links and zero-capacity links, then obtains up to four simple (no repeated node) paths with NetworkX's weighted `shortest_simple_paths` routine, which generates paths in increasing weighted length [5]. The edge cost is

$$w(e) = p(e) + \beta \cdot L(e) / C(e), \quad \beta = 1.0 \tag{1}$$

where $p(e)$ is the configured propagation latency (ms), $L(e)$ the current offered load (Mbps) and $C(e)$ the link capacity (Mbps). This is a heuristic combining latency and utilization, not a

measured end-to-end delay. A seeded permutation is applied when several candidates exist, so a candidate index is not a permanent ranking by cost.

The scorer uses two GATv2 layers, each with four attention heads of 16 features, producing a 64-feature node representation [3]. Flow-conditioned pooling combines node representations along each candidate path, and the scorer also receives source and destination representations, flow features and the six candidate features. The frozen configuration uses a dueling output architecture that separates a state-value estimate from action-advantage estimates [6]; the mean advantage is computed over valid candidates:

$$Q(s,a) = V(s) + A(s,a) - \text{mean}_{*b* \in \text{valid}}\, A(s,b) \quad (2)$$

Invalid candidates receive masked scores and cannot win normal selection. The environment has an additional idle/no-route sentinel at action K, but the network scores only the K candidate actions; deliberately masked nonfinite scores are therefore distinct from a nonfinite score on a valid candidate.

The inference artifact is the frozen step-0 checkpoint transferred from behavior-cloning initialization before online reinforcement-learning updates. The repository's reinforcement-learning framework does not establish that this checkpoint gained performance through online learning. During inference the weights remain fixed and NoisyNet layers use evaluation behavior; repeated observations update the routing input, not the learned weights.

4) Decision Validation and Fallback

The normal decision is the highest-scoring valid candidate. The wrapper then checks for no valid candidate, nonfinite valid scores, an invalid action, an unavailable selected path, and the defined overload condition, which requires the selected route's predicted maximum backbone utilization to exceed 1.0 while another available valid candidate remains below 0.9. This is a specific trigger, not a guarantee that every selected route stays below capacity. When a check fails and fallback is enabled, the wrapper searches the full available graph using load-aware Dijkstra with $\alpha = 4.0$ and link cost

$$c(e) = \max\{\, \ell(e) \cdot [1 + \alpha\, \rho(e)],\, 10^{-12} \,\} \quad (3)$$

where $\rho(e) = u(e) / (1 - \min(u(e), 0.95))$, $u(e)$ is current offered-load utilization and $\ell(e)$ is current modeled link latency. Failed elements are excluded. Because fallback searches the full graph, its route need not belong to the candidate set, and a failed fallback can return no route. Fig. 4 shows the ordered checks; the returned decision records whether fallback was invoked and its reason.

5) Flow Progression, Metrics and Reward

Simulation advances in decision steps, with one new request per nonterminal step. Despite the legacy class name `PoissonTrafficGenerator`, the current implementation does not draw a Poisson-distributed number of arrivals; requests have seeded endpoints, offered demand and geometrically distributed lifetimes, and traffic intensity is calibrated through concurrent flow duration and demand. The API also adjusts its preset calibration using host-access capacity where applicable. A successful route installs the full offered demand on each path link, which can overload a link: routing does not enforce strict capacity admission. Expired flows release their loads. A link or router failure invalidates and releases affected active flows without rerouting them, while a bandwidth drop leaves active flows installed and changes the conditions against which their load is evaluated (Fig. 3).

Latency and loss are model outputs. The delay calculation uses link capacity and offered load with a 12,000-bit packet-size assumption; away from saturation its queue-related term is $1000/(\mu - \lambda)$ ms, where μ and λ are the service and arrival rates in packets per second (an M/M/1 system-time expression that includes service time). The term is capped at 10,000 ms near saturation and propagation latency is added. Path loss combines link-loss probabilities as one minus the product of survival probabilities, and the API combines configured base loss with congestion loss. Delivered demand is limited by demand and the path's available capacity before the new flow is installed; it is not a packet count and is not multiplied by path survival probability. All of these are simulation approximations, not measurements from a physical network.

The reward combines QoS-sensitive latency and loss costs, bandwidth credit, minimum-bandwidth cost and a backbone-utilization cost; Table III preserves the source definitions [8]. The flap term is applied separately. The environment returns an unclipped scalar reward, while a separate wrapper can scale and clip rewards for training; the latency and utilization caps limit components or inputs, not the final reward. The existence of a reward function describes the training framework and does not establish online training of the frozen checkpoint.

TABLE III
REWARD, CONGESTION AND CLIPPING (SOURCE TABLE UNCHANGED [8])

Component	Per-step contribution before the common QoS and priority factors
Latency	−latency_sensitivity × path_latency_ms / latency_scale_ms
Loss	−loss_sensitivity × path_loss / loss_scale
Bandwidth	bandwidth_credit
Minimum bandwidth	−max(0, min_bandwidth_mbps − delivered_mbps) / target_bandwidth_mbps
Utilization	−utilization_weight × min(max_path_backbone_utilization, 2.0)
Flap	The existing separate flap_penalty when the same-flow route changes without meaningful quality improvement

6) Application Integration and Recorded Outputs

Fig. 5 maps the application to its source modules. The backend stores simulation-result JSON through its persistence layer. The current simulation API emits AI decision traces containing candidates, scores, chosen paths and fallback reasons alongside aggregate metrics, and frontend result and explainability views read them when available. This capability differs from older documentation describing aggregate-only outputs and does not prove that earlier saved evaluations contain traces. The individual routing call can return attention weights, but the simulation trace does not establish saved attention attribution or a complete history of per-link heatmaps.

D. Diagrams

Figs. 1–5 are drawn from the inspected implementation and describe simulation and application behavior. Flow allocation

is the project's representation of forwarding; no diagram implies packet transmission through physical devices or an SDN controller.

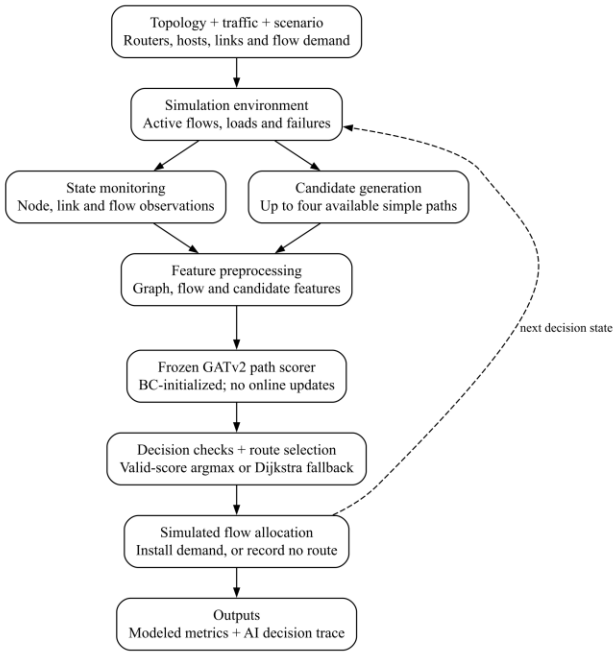


Fig. 1. System architecture: simulator observations feed a frozen path scorer and validation wrapper; selected flows update simulator state. The dashed feedback arrow denotes changes to simulator state, not online training. BC: behavior cloning.

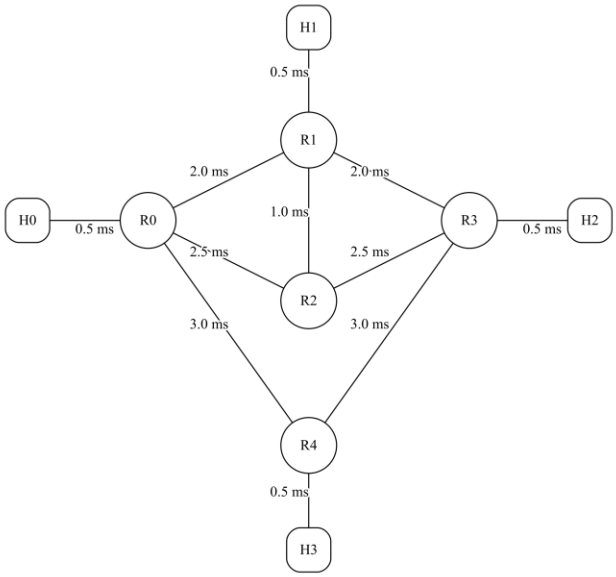


Fig. 2. Network topology: supplied five-router, four-host example (nine nodes, eleven undirected links); all links 1,000 Mbps with 0% configured loss; labels are propagation latencies in ms. H0 to H2 is an illustrative endpoint choice with alternatives H0–R0–R1–R3–H2 and H0–R0–R2–R3–H2, neither a measured winning route.

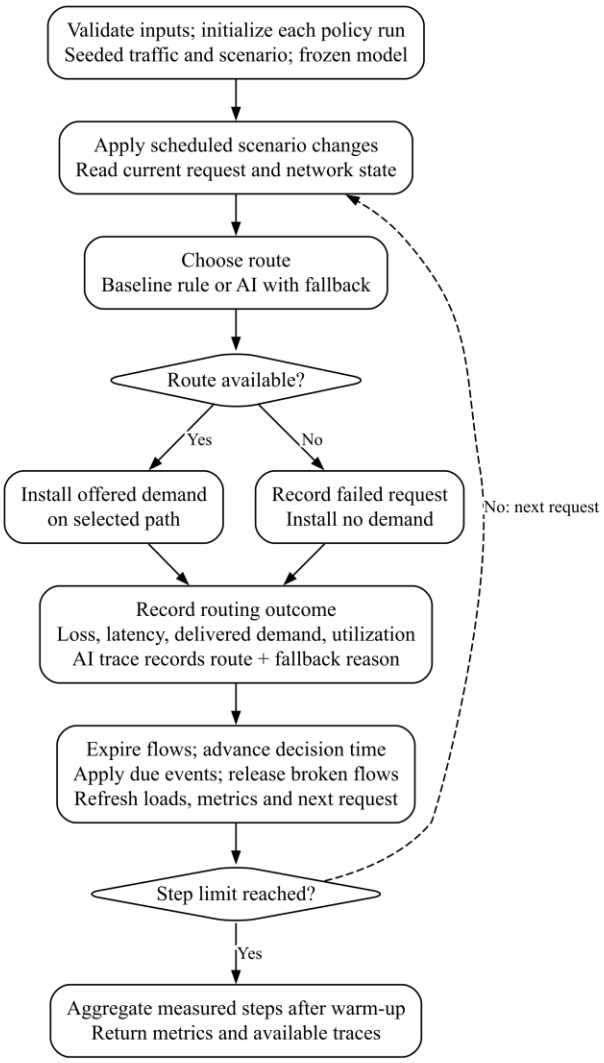


Fig. 3. End-to-end workflow: route selection, simulated allocation, outcome recording and state progression until the configured step limit. Warm-up exclusion applies to the aggregate measurement period only; state is still updated during warm-up.

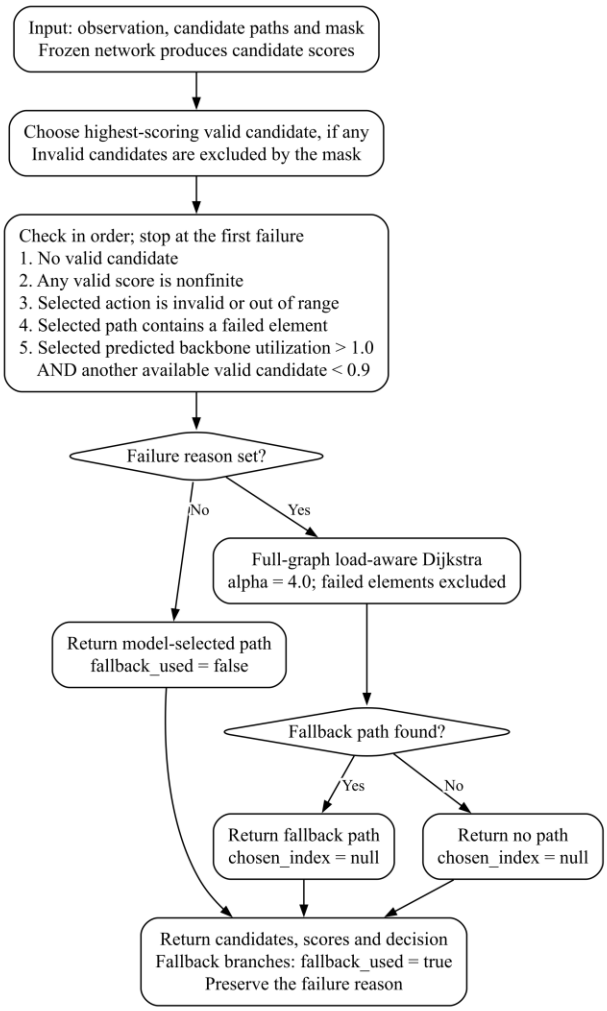


Fig. 4. Routing decision flowchart: masked score selection and ordered fallback checks implemented in models/safety_fallback.py, with fallback enabled. A fallback result retains its failure reason even if Dijkstra finds a valid replacement route.

III. Implementation

A. Technologies Used

The implementation is built with the technologies summarized in Table I (Section II-B): Python, PyTorch and PyTorch Geometric for the learned scorer, Gymnasium and NetworkX for the simulator, FastAPI with SQLAlchemy for the backend, and React with TypeScript for the frontend.

1) Source Organisation and Application Integration

The implementation is organised into three directories: `AI/` (environment, routing policies, model, training utilities, topology datasets and evaluation outputs), `backend/` (FastAPI services) and `frontend/` (React/TypeScript configuration and result views). The adapter in `backend/app/services.py` translates frontend topology and flow fields into the model API schema, because interface device types and link fields differ from the simulator's router/host representation. It invokes `AI/models/api.py` and does not train a model per request. The expected frozen-model hash is checked before use; the inspected checkpoint has SHA-256 `45957597f56cf163c5ae5d69f6fe6959ddb638601c7ade97b865a848151fff28`, matching the recorded final-test artifact.

Application handlers in `backend/app/main.py` coordinate simulation requests and store result JSON using the SQLAlchemy models, and frontend calls pass through `frontend/src/api/client.ts`. This describes the inspected source path and does not establish a successful deployment against the configured PostgreSQL/Neon database.

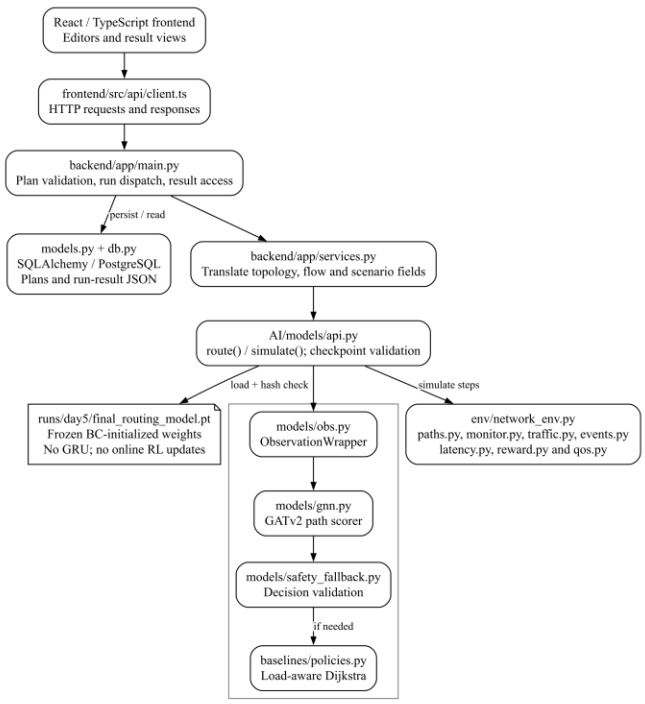


Fig. 5. Implementation architecture: source-module relationships, frozen-checkpoint loading and simulation dependencies. Paths inside the AI component are relative to AI/. Database connectivity is not demonstrated by the diagram.

2) Routing a Single Flow

The public `route(topology_json, link_state, history, flow)` function validates the topology and flow, applies the supplied link state, constructs the routing environment and produces an observation. The observation wrapper prepares graph, flow and candidate features, the frozen model scores the candidates, and `choose_with_fallback` applies the checks of Section II-C4. The following excerpt from `AI/models/api.py` (re-wrapped for column width) shows the preprocessing and fallback call; it is source code, not a newly executed output.

```
wrapper = ObservationWrapper(
    env, graph.graph.get("name", "user-topology"))
state = wrapper.process(observation)
# The frozen checkpoint has no GRU. History is
# accepted for forward compatibility; the current
# feed-forward model does not consume it.
_ = history
decision = choose_with_fallback(
    _load_agent(), env, state,
    np.random.default_rng(0), enabled=True)
```

The response contains candidate paths and scores, the chosen index and path, the fallback flag and reason, and available attention weights; nonfinite scores are serialised as null. Scores rank actions and are not routing-success probabilities. A fallback path can lie outside the candidate set, in which case the chosen candidate index is null, and if no route can be found the returned path is also null.

3) Simulation and Measurement

The environment follows Gymnasium's reset/step interface, in which `reset()` returns an initial observation and information and `step()` returns an observation, reward, termination and truncation flags and information [7]. The project uses the information fields to record route outcomes and link-utilization measures, and reaching the configured decision-step limit truncates an episode. Each policy operates on its own evolving environment: shared request sequences make offered traffic comparable, but route choices produce different loads and later observations, so equal request hashes do not imply identical network states.

The evaluation code assigns loss 1.0 and delivered demand 0.0 to an unrouted request; for a routed request it records modeled path loss, path latency and delivered demand. Reported mean loss therefore includes routing failures, whereas mean and percentile latency are computed over routed requests only and must be read alongside failure and loss measures. Delivered throughput is the mean delivered-demand value in Mbps, not a packet count or measured goodput. The saved datasets also contain backbone-utilization, route-change, decision-time and recovery fields. The flapping statistic compares successful routes for repeated source–destination pairs and is not a count of reroutes of an active flow, since broken flows are released rather than rerouted; recovery is a loss-based diagnostic in decision steps, not a routing-protocol convergence time.

4) Model Provenance and Testing Approach

The frozen inference artifact was selected at step 0 after shape-compatible transfer from behavior cloning. The repository includes Double-DQN training [9], dueling outputs, NoisyNet layers [11], prioritized replay [10] and other study variants, but their presence does not change the artifact's provenance. The final test evaluates the frozen scorer together with its fallback wrapper. The Day 5 report records a historical test-suite result of 174 passed, 1 warning and a checkpoint-reload difference of 0.0; these are supplied records, not tests rerun here. Backend test sources cover authentication, ownership checks, topology translation, result filtering and database configuration using an isolated SQLite fixture, which does not establish current pass status or live PostgreSQL compatibility. Verification here instead checks the supplied evidence directly: checkpoint hashing, episode counts and unique keys, agreement between CSV/JSON/JSONL files, paired request hashes and event schedules, recomputation of reported means and sample standard deviations, and recalculation of acceptance criteria. No training, final-test simulation or database migration was performed.

B. Screenshots and Outputs

No genuine application screenshots were found among the supplied assets, so none are fabricated. The saved checkpoint_load_check.json provides the compact masking and checkpoint-reload diagnostic below.

```
{
  "validation_graph": "Arpanet19728",
  "valid_actions": 4,
  "invalid_actions": 0,
  "valid_q_finite": true,
  "invalid_q_negative_inf": true,
  "mask_probe_invalid_actions": 1,
  "mask_probe_invalid_q_negative_inf": true,
  "reload_max_abs_q_difference": 0.0
}
```


Current simulation code can return per-decision traces, but the final retry dataset contains episode summaries and aggregate diagnostic counts, not those traces. It therefore cannot supply authentic per-step Q-value screenshots, complete per-link heatmaps or saved attention-attribution figures for the final test. Detailed validation and final-test results are presented in Section IV and its per-graph tables.

IV. Evaluation and Results

This section reports the saved validation benchmark and the completed locked-test records. All values are simulator outputs. No episode was rerun for this report revision. Packet loss is shown as a percentage; throughput is modeled delivered demand in Mbps, not counted packets or measured goodput.

A. Frozen Model Architecture

The frozen checkpoint uses two edge-aware GATv2 layers with four heads of 16 channels, giving 64-dimensional node embeddings. A path-attention scorer combines each candidate path with endpoint, flow and candidate features. A dueling head forms Q-values from a state value and action advantages; invalid candidates are masked before selection. The safety wrapper runs after scoring. The auxiliary head predicts next-state backbone utilization during training only, and the frozen checkpoint has no GRU.

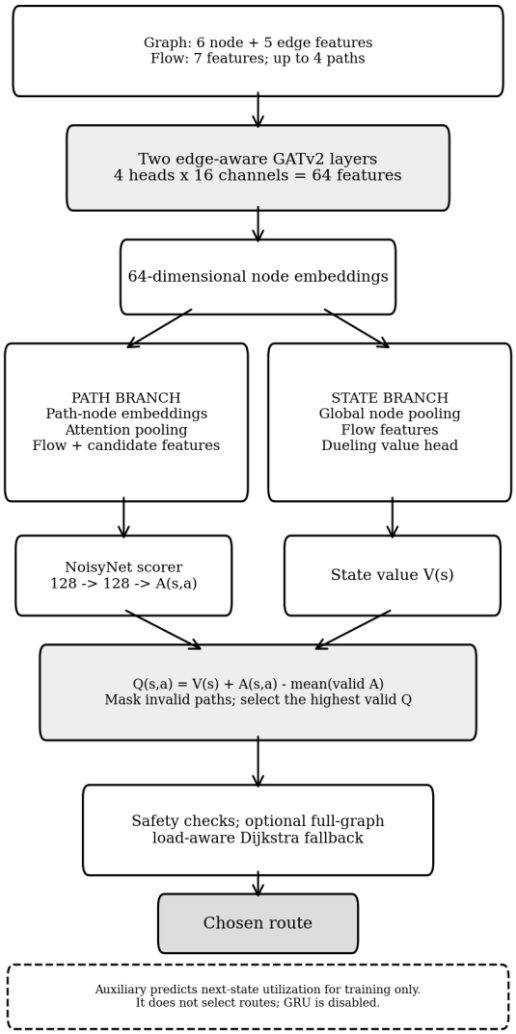


Fig. 6. Frozen GATv2 path scorer and the separate safety wrapper. The auxiliary utilization head is training-only; the frozen checkpoint has no GRU.

The behavior-cloning initialization used 100,000 decisions labeled by dijkstra_load_aware_k. It trained on 88,167 decisions and held out 11,833 from graph-seed blocks across the 12 training topologies; weighted held-out top-1 expert agreement was 93.26%. The held-out blocks test separation by graph-seed block, not transfer to unseen topologies. The saved FULL checkpoint was selected at training step 0, before online Double-DQN updates; consequently, this report does not attribute its validation or test results to successful online reinforcement-learning improvement.

B. Evaluation Protocol and Run Integrity

The validation benchmark used five validation graphs, seeds 1000-1004, five scenarios and 500 decision steps per episode, with the first 100 steps treated as warm-up. The saved validation summary contains 375 policy episodes (25 graph-scenario-seed combinations x three policies). The locked test matrix used five test graphs, five seeds, five scenarios, three intensities (0.05, 0.10 and 0.20), and eight policies, yielding 3,000 completed episode rows. Each episode used 500 decision steps and 100 warm-up steps. Pooled test rows below average 25 graph-seed episodes per scenario, intensity and policy.

The pre-registered criteria were committed before the first test run. The initial attempt began on 13 October 2026 and was interrupted during laptop shutdown; its run protocol records about 1,475 of 3,000 jobs, while the shutdown checkpoint records 1,450 durable jobs. A user-authorized same-settings retry began on 18 October and completed all 3,000 rows. Fifty-nine overlapping record keys were available for comparison: request-sequence hashes, event schedules and routing outcome metrics matched, while measured decision time differed. Thus the duplicate overlap reproduces the routing outcomes, not every timing field. The report uses the completed retry dataset and does not treat the interrupted attempt as an independent run.

The results were checked from the saved CSV, JSON and JSONL records; no simulations, model training or tuning were performed for this revision. Earlier calibration included two designated test graphs, so the final test does not support a claim of untouched test-topology generalization.

C. Validation Results

Fig. 7 summarizes validation packet loss, latency, delivered demand and high-utilization frequency across five scenarios. Table IV consolidates packet-loss, latency and delivered-demand means with sample standard deviations.

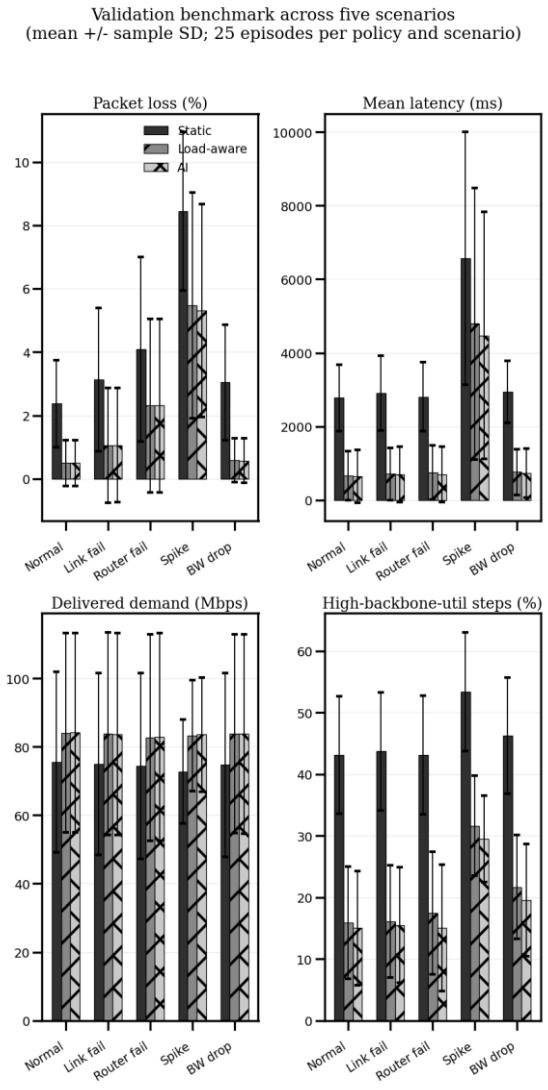


Fig. 7. Validation packet loss, latency, delivered demand and high-utilization frequency across five scenarios (mean +/- sample SD).

TABLE IV

VALIDATION PERFORMANCE ACROSS FIVE SCENARIOS

Scenario / policy	Packet loss (%) mean ± SD	Latency (ms) mean ± SD	Delivered demand (Mbps) mean ± SD
Normal / Static Dijkstra	2.3702 +/- 1.3712	2781.4 +/- 900.5	75.47 +/- 26.42
Normal / Load-aware Dijkstra	0.4971 +/- 0.7180	669.4 +/- 665.9	84.12 +/- 29.11
Normal / AI + fallback	0.4960 +/- 0.7207	647.3 +/- 713.7	84.12 +/- 29.18
Link failure / Static Dijkstra	3.1291 +/- 2.2588	2909.9 +/- 1014.9	74.96 +/- 26.53
Link failure / Load-aware Dijkstra	1.0566 +/- 1.8061	707.1 +/- 705.0	83.82 +/- 29.63
Link failure / AI + fallback	1.0573 +/- 1.7997	701.0 +/- 749.0	83.74 +/- 29.59
Router failure / Static Dijkstra	4.0877 +/- 2.9034	2808.1 +/- 935.8	74.42 +/- 27.12
Router failure / Load-aware Dijkstra	2.3136 +/- 2.7365	752.6 +/- 733.7	82.73 +/- 30.12
Router failure / AI + fallback	2.3092 +/- 2.7392	699.5 +/- 755.3	82.85 +/- 30.32
Traffic spike / Static Dijkstra	8.4369 +/- 2.5018	6564.4 +/- 3432.9	72.77 +/- 15.18
Traffic spike / Load-aware Dijkstra	5.4742 +/- 3.5662	4790.6 +/- 3683.6	83.28 +/- 16.29
Traffic spike / AI + fallback	5.3132 +/- 3.3546	4473.4 +/- 3349.0	83.59 +/- 16.63
Bandwidth drop / Static Dijkstra	3.0443 +/- 1.8209	2945.4 +/- 843.6	74.71 +/- 26.91
Bandwidth drop / Load-aware Dijkstra	0.5824 +/- 0.6923	764.4 +/- 624.3	83.81 +/- 29.10
Bandwidth drop / AI + fallback	0.5753 +/- 0.6941	731.3 +/- 672.0	83.81 +/- 29.12

The AI-with-fallback validation means are close to load-aware Dijkstra in several metrics; they are not uniformly better. For example, the AI has slightly higher mean loss

under link failure, while its mean latency is lower in the saved traffic-spike comparison. These are comparisons within this simulator and these five validation graphs.

The saved paired wrapper check compared the same frozen policy with fallback disabled and enabled on the five validation graphs and five scenarios. Mean loss did not worsen in any scenario; the recorded non-regression check passed its 5% relative limit. The test matrix has no separate AI-without-fallback arm, so a paired wrapper effect is available for validation only.

Across 62,500 validation decisions, fallback triggered 229 times for no valid candidate and 382 times for an overloaded candidate (611 total, 0.98%). No invalid-action, failed-path or nonfinite-Q trigger was recorded. The wrapper-trigger counts are a mechanism diagnostic, while the loss comparison above is the performance check.

D. Locked Test Results and Pre-registered Criteria

Fig. 8 makes the normal-scenario T1 result visible. At intensity 0.05, the loss reduction is just below the required 50%; the two higher tested intensities exceed it. Fig. 9 shows normal-scenario mean packet loss at intensity 0.10 by test graph and policy.

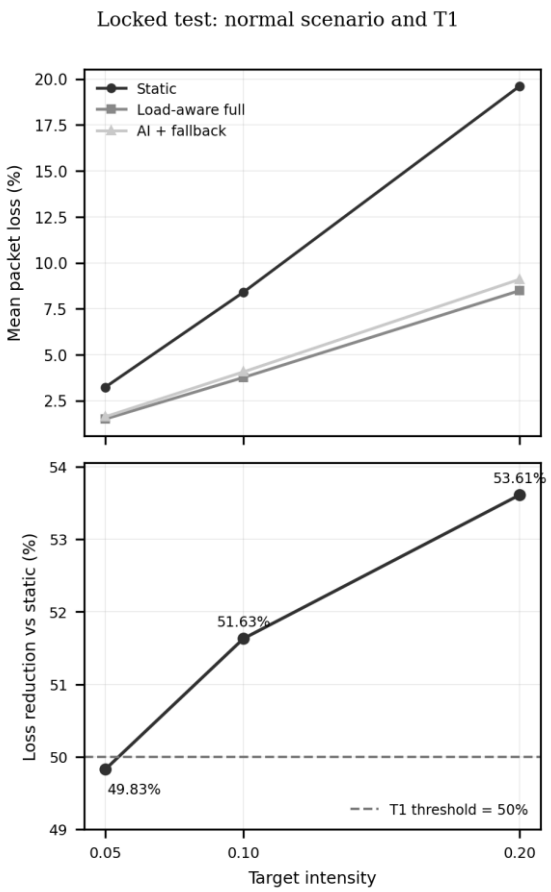


Fig. 8. Normal-scenario pooled test loss and percentage reduction versus static Dijkstra; the dashed line marks the 50% T1 threshold.

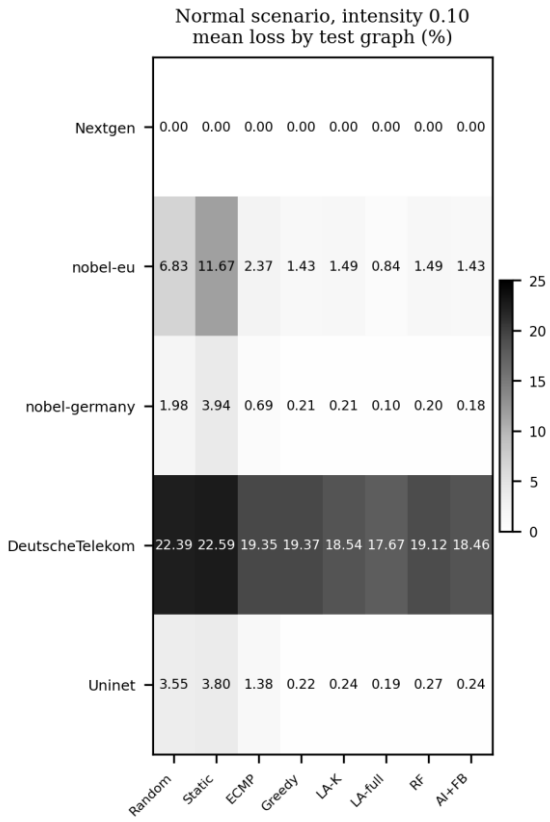


Fig. 9. Normal-scenario mean packet loss (%) at intensity 0.10 for each locked test graph and all eight policies; values average five seeds.

TABLE V
POOLED TEST PACKET LOSS BY SCENARIO AND INTENSITY (%)

Scenario	I	Static	Load-aware	AI + fallback
Normal	0.05	3.2297	1.4977	1.6205
Normal	0.10	8.3994	3.7597	4.0629
Normal	0.20	19.6231	8.4892	9.1030
Link failure	0.05	3.2976	1.5529	1.6813
Link failure	0.10	8.6780	4.0688	4.3420
Link failure	0.20	20.2621	9.3789	9.9555
Router failure	0.05	4.8192	3.0909	3.2106
Router failure	0.10	10.1917	5.6611	5.9651
Router failure	0.20	21.3618	10.6669	11.1771
Traffic spike	0.05	7.1840	4.6440	4.6544
Traffic spike	0.10	13.6994	7.7675	8.0215
Traffic spike	0.20	26.8765	14.7768	15.1301
Bandwidth drop	0.05	3.5706	1.6171	1.7042
Bandwidth drop	0.10	9.0344	3.9108	4.1924
Bandwidth drop	0.20	20.6699	8.7484	9.3967

TABLE VI
PRE-REGISTERED TEST CRITERIA STATUS

Criterion	Status	Evidence
T1	FAIL	At intensity 0.05, AI loss reduction is 49.83%, below 50%; 0.10 and 0.20 pass.

Criterion	Status	Evidence
T2	PASS	All 15 scenario-intensity groups satisfy max(1.15 x load-aware loss, load-aware loss + 0.002).
T3	PASS	3,000 rows completed; 0 accepted invalid actions, 0 nonfinite valid-Q decisions, and 0 nonfinite metric values; no recorded crash.
T4	REPORTED	Aggregate wins/ties/losses are in Table VII. The main report omits the full 75-row breakdown; saved evaluation records retain it.
T5	REPORTED	84.47% overall expert agreement; graph-level agreement is in Table VIII.

At all three normal-scenario intensities, static loss exceeds the pre-registered threshold, so T1 applies. T1 is the only failed pass/fail performance gate: the reduction is 49.83% at intensity 0.05, then 51.63% and 53.61% at 0.10 and 0.20. T4 and T5 are reporting requirements, not performance pass/fail thresholds. For T4, ties use equality at six-decimal precision in the saved records; the shortened report shows aggregate wins/ties/losses and one per-graph operating-point highlight, while the complete 75-group data remain in the saved run records.

The T5 agreement is a decision-level diagnostic, not model accuracy and not proof of lower packet loss. The graph-level agreement percentages in Table VIII are pooled from the saved per-episode counts.

Fallback was invoked on 1,659 of 187,500 AI decisions (0.885%). The two triggers were an overloaded candidate and no valid candidate. The trigger counts alone do not show that every fallback found a route.

TABLE VII
AI-WITH-FALLBACK WINS, TIES AND LOSSES OVER 75 T4 GROUPS

Baseline	AI wins	Ties	AI losses
Random	69	6	0
Static	68	7	0
ECMP	68	7	0
Greedy	40	8	27
Load-aware K	55	11	9
Load-aware full	13	11	51
Random forest	54	8	13

E. Per-Graph T4 Losses

Table VIII is a focused per-graph snapshot for the normal scenario at intensity 0.10. It shows packet-loss change versus static Dijkstra and T5 agreement; it is a selected-condition highlight, not the full test matrix. The pooled test table above still covers all five scenarios and all three intensities.

TABLE VIII
TEST GRAPH HIGHLIGHTS: NORMAL SCENARIO, INTENSITY 0.10

Test graph	Routers	Static loss (%)	AI loss (%)	Reduction vs static	T5 agreement
Nextgen	17	0.00	0.00	— (both zero)	88.44%
nobel-eu	28	11.67	1.43	87.73%	86.84%
nobel-germany	17	3.94	0.18	95.46%	87.42%
DeutscheTelekom	30	22.59	18.46	18.27%	72.99%
Uninet	13	3.80	0.24	93.59%	86.65%

Training-stability status is separate from the locked test gates. In the corrected 1,200-step FULL smoke, |Q| p99.9 was 60.2468 and the maximum was 60.2998, against the unchanged p99.9 < 60 and maximum < 100 thresholds; therefore, the p99.9 gate failed and the maximum gate passed. The reported p99.9 is NumPy's 99.9th percentile over 250 per-update maxima of absolute Q-values, not over individual actions or validation decisions. Validation loss at steps 800, 1,000 and 1,200 was 0.012613, 0.013707 and 0.020629, respectively. The frozen checkpoint remains the step-0 selection; this evidence does not establish that online RL improved the policy.

V. Conclusion

The project implements a flow-level network-routing simulator connected to a web application. It represents network topology, concurrent traffic and changing link conditions, constructs candidate paths, and uses a frozen GATv2-based model to rank them; a separate wrapper checks the decision and can invoke full-graph load-aware Dijkstra. These components support routing experiments in software, while physical packet forwarding and live-router control remain outside the demonstrated implementation. The selected model was transferred from behavior-cloning initialization and frozen at step 0, before online reinforcement-learning updates, so the evaluated checkpoint does not establish gains from those updates. Its decisions depend on the current processed observation, with no active recurrent-history module.

The saved final test contains 3,000 episodes across five test graphs, three intensities, five scenarios, five seeds and eight policies. The CSV, JSON and JSONL records agree, and request hashes and event schedules match within the policy-comparison groups; this establishes consistency of the supplied records, but the experiment was not independently rerun. Under normal conditions, AI with fallback reduced mean modeled loss relative to static Dijkstra by approximately 49.83%, 51.63% and 53.61% at intensities 0.05, 0.10 and 0.20. The criterion requiring at least a 50% reduction therefore failed overall because the lowest-intensity result fell short. The combined policy passed the specified tolerance relative to load-aware Dijkstra in all 15 scenario–intensity groups, but its mean loss was higher than that baseline’s in every group. The evidence supports improvement over the static comparator under the

tested conditions without establishing superiority over full-graph load-aware routing.

The recorded AI episodes contained no accepted invalid actions or nonfinite valid-score decisions, and fallback was invoked in 1,659 of 187,500 decisions, including warm-up. These observations describe the combined model-and-wrapper system; they do not show that fallback always found a route or isolate how much performance came from the neural scorer alone. The demonstrated contribution is an inspectable routing workflow with candidate scoring, explicit fallback rules and saved comparative evaluation. Its limits remain material: delay and loss are simulated, hardware requirements have not been established experimentally, a live application/database deployment has not been verified, and earlier calibration included two designated test graphs. The full validation tables, locked-test plots, acceptance-criteria status, run protocol and per-graph comparisons are reported in Section IV.

VI. Future Scope

The following are proposals, not claims about completed functionality or guaranteed performance gains.

Experimental separation and replication. Reserve a new set of topologies before calibration or model selection and keep it outside exploratory analysis; repeat comparisons across more seeds, traffic patterns and topology families; record checkpoint hash, software environment, source revision, configuration and interruption history; and report paired uncertainty estimates rather than rounded means alone.

Separating scorer and fallback. Compare the final-test policy with the same frozen scorer without fallback under an approved protocol, vary the candidate limit, give competing policies the same candidate set, and record fallback success and failure separately from invocation counts.

Controlled learning and temporal models. Investigate further reinforcement-learning updates only as a new model-development stage with the frozen artifact retained as reference and explicit acceptance criteria; evaluate a recurrent model on the exposed observation histories against the current-observation model.

Richer traffic and failure modeling. Introduce variable arrival rates, packet-level queues, retransmission behavior and active-flow rerouting (with a defined policy for releasing old allocations and handling disruption), evaluated separately and compared with a packet-level simulator or isolated testbed.

Trace persistence and deployment verification. Extend decision-trace support with a versioned result schema, complete per-link histories and identified attention outputs; demonstrate database persistence, retrieval and frontend rendering with genuine captured outputs; and, before any live-network integration, add a telemetry interface, a forwarding-control mechanism and tested handling of stale, missing or invalid inputs.

Scalability and resource measurement. Measure decision latency, memory and simulation runtime as graph size, candidate count and concurrent load increase, to derive evidence-based hardware requirements and identify the dominant cost. Tests beyond the 5–50-router API range would require revisiting input limits and evaluating policy behavior on larger graphs.

Acknowledgment

The authors thank Pragya Vats for serving as the guide for this mini project and acknowledge UnitedWorld Institute of Technology, Karnavati University, where this work is submitted as part of the B.Tech. Computer Science programme. The authors also acknowledge the researchers and open-source contributors whose networking datasets, machine-learning libraries and software tools support the implementation and its evaluation. The interpretation of the recorded results and any remaining errors are the authors' responsibility.

References

[1] J. Moy, "OSPF version 2," Internet Eng. Task Force, RFC 2328, Apr. 1998, doi: 10.17487/RFC2328. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2328.html

[2] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2260–2270, 2020, doi: 10.1109/JSAC.2020.3000405. [Online]. Available: https://arxiv.org/abs/1910.01508

[3] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022, arXiv:2105.14491, doi: 10.48550/arXiv.2105.14491. [Online]. Available: https://arxiv.org/abs/2105.14491

[4] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, ser. Proc. Mach. Learn. Res., vol. 15, 2011, pp. 627–635. [Online]. Available: https://proceedings.mlr.press/v15/ross11a.html

[5] NetworkX Developers, "shortest_simple_paths," NetworkX 3.7 Documentation. [Online]. Available: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.simple_paths.shortest_simple_paths.html. Accessed: Oct. 28, 2026.

[6] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., vol. 48, 2016, pp. 1995–2003. [Online]. Available: https://proceedings.mlr.press/v48/wangf16.html

[7] Farama Foundation, "Env," Gymnasium Documentation. [Online]. Available: https://gymnasium.farama.org/api/env/. Accessed: Oct. 28, 2026.

[8] LAN Lords project, "Source tree (AI/, backend/, frontend/), environment specification (AI/docs/env_spec.md), Day 5 final model freeze and benchmark handoff (AI/outputs/day5_final/), final retry evaluation records (AI/outputs/day6_7_lite/test_final_retry/), and topology and calibration sources (AI/examples/topology.json, AI/data/splits.json, AI/outputs/calibration_v2.csv)," unpublished project materials, inspected Oct. 28, 2026.

[9] H. van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-Learning," in Proc. 30th AAAI Conf. Artificial Intelligence, 2016, pp. 2094–2100, doi: 10.1609/aaai.v30i1.10295. Available: https://ojs.aaai.org/index.php/AAAI/article/view/10295

[10] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," in Proc. Int. Conf. Learning Representations (ICLR), 2016. Available: https://arxiv.org/abs/1511.05952

[11] M. Fortunato et al., "Noisy Networks for Exploration," in Proc. Int. Conf. Learning Representations (ICLR), 2018. Available: https://arxiv.org/abs/1706.10295